

# Compound Interest Calculator

AWS Serverless Architecture

Request-to-Response Flow & Sequence Diagram

## Step-by-Step Walkthrough

### 1. User Opens the App

Browser loads the React SPA from **CloudFront** (backed by S3). On mount, the frontend opens a **WebSocket connection** to `wss://<api-id>.execute-api.us-east-1.amazonaws.com/prod`.

### 2. WebSocket `$connect`

API Gateway receives the upgrade request and invokes the **API Lambda** with `routeKey: "$connect"`. The Lambda logs the connection and returns `{ statusCode: 200 }`. API Gateway completes the WebSocket handshake; the client is now connected.

### 3. User Submits a Calculation

The user fills in **principal**, **annualRate**, **years**, and **compoundingFrequency** and clicks "Calculate". The frontend sends a WebSocket message:

```
{
  "action": "calculate",
  "principal": 10000,
  "annualRate": 5.5,
  "years": 10,
```

```
"compoundingFrequency": 12
}
```

#### 4. API Lambda Processes the `calculate` Route

API Gateway matches `action: "calculate"` and invokes the **API Lambda**, which:

1. Extracts `connectionId` and `domainName/stage` from `requestContext`
2. Parses and validates the request body
3. Generates a unique `calculationId` (UUID)
4. Builds the `wsCallbackUrl`
5. Constructs a Kafka event with all inputs + `connectionId` + `wsCallbackUrl`
6. **Publishes the event to Kafka** topic `calculation-requests`
7. Returns `{ statusCode: 200, body: { calculationId, status: "PENDING" } }`

#### 5. Kafka Delivers the Message

Amazon **MSK** receives the message on topic `calculation-requests`. The MSK event source mapping triggers the **Calculator Lambda** with a batch of records.

#### 6. Calculator Lambda Computes the Result

The Lambda deserializes each Kafka record, extracts inputs, and applies the compound interest formula:  $A = P \times (1 + r/n)^{n \times t}$ . Builds a result payload with `status: "COMPLETED"` and `finalAmount`.

#### 7. Calculator Lambda Pushes Result via WebSocket

Using the `connectionId` and `wsCallbackUrl` from the Kafka event, the Lambda calls `postToConnection(connectionId,`

resultPayload) via the **API Gateway Management API**.

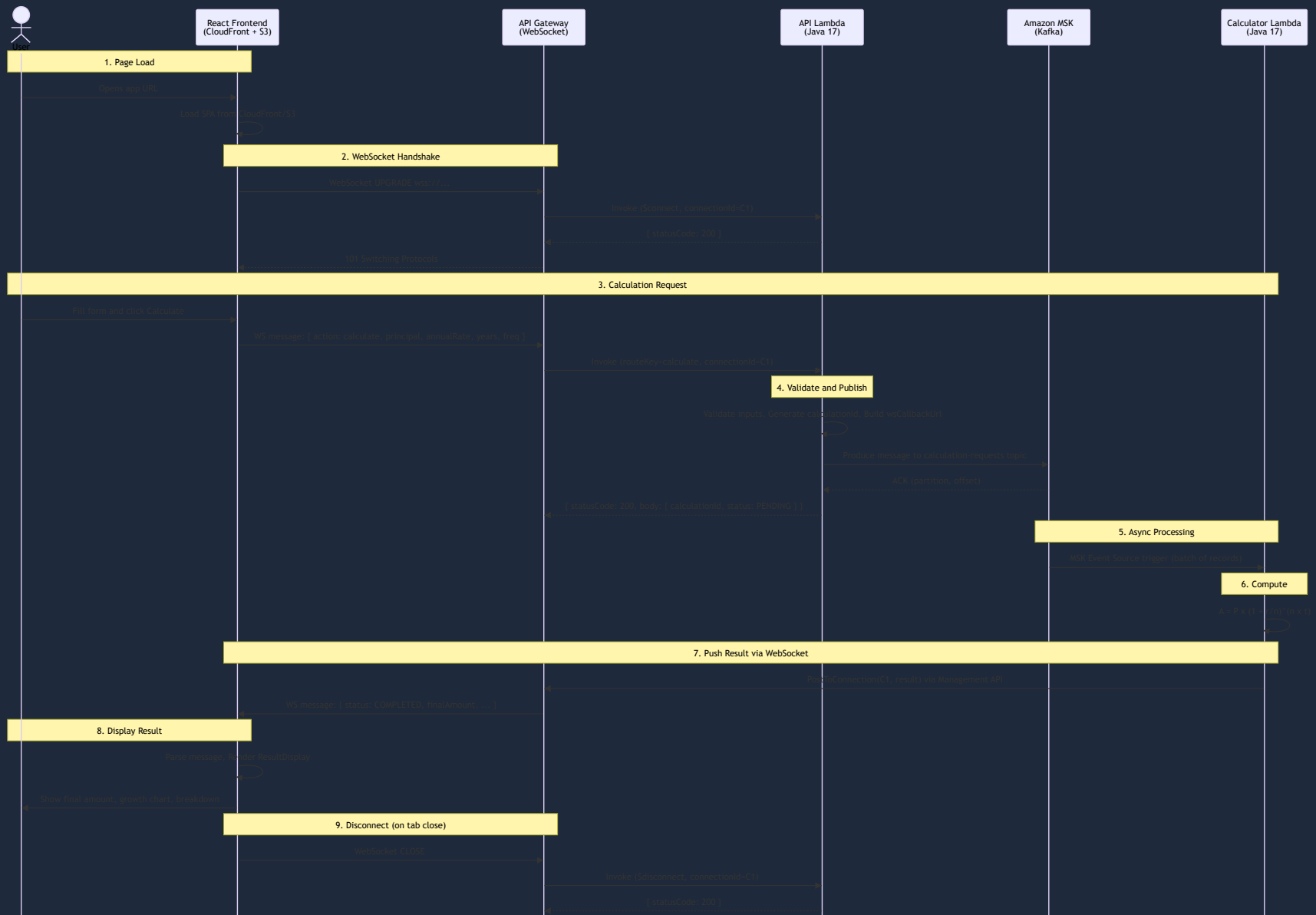
## 8. Frontend Displays the Result

The frontend's `onmessage` handler receives the JSON payload, detects `status: "COMPLETED"`, and renders the **ResultDisplay** component with final amount, growth chart, and year-by-year breakdown.

## 9. WebSocket `$disconnect`

On tab close or timeout, API Gateway sends a `$disconnect` event to the API Lambda. The Lambda logs the disconnection and returns `{ statusCode: 200 }`.

## Sequence Diagram



## Component Summary

| Component         | Technology                 | Role                                                            |
|-------------------|----------------------------|-----------------------------------------------------------------|
| Frontend          | React + Vite + TailwindCSS | SPA served from CloudFront/S3; WebSocket client                 |
| API Gateway       | AWS WebSocket API          | Routes \$connect, \$disconnect, calculate; manages connections  |
| API Lambda        | Java 17, Kafka Producer    | Validates input, publishes to Kafka with WS callback info       |
| Amazon MSK        | Apache Kafka 3.5.1         | Decouples request ingestion from computation                    |
| Calculator Lambda | Java 17, MSK Consumer      | Computes compound interest, pushes result via WS Management API |
| CloudFront + S3   | AWS CDN + Object Storage   | Serves static frontend assets globally                          |

### Key Design Decisions

- **Kafka as decoupler:** The API Lambda returns immediately after publishing, keeping WebSocket response times fast. The heavy computation happens asynchronously.
- **WebSocket push (not polling):** The Calculator Lambda pushes results directly to the client's open connection — no polling, no DynamoDB, no wasted API calls.
- **Stateless Lambdas:** No shared state between Lambdas. The `connectionId` and `wsCallbackUrl` travel through Kafka so the Calculator Lambda knows where to send results.